

Sportbooking

TECHNICAL DOCUMENTATION

ENRICO BARBATANO

Sommario

1 SRS (Software Requirements Specification).....	2
1.1 Introduzione.....	2
1.1.1 Scopo del documento	2
1.1.2 Descrizione generale del sistema	2
1.1.3 Hardware and Software Requirements	2
1.1.4 Related Systems	3
1.2 User Stories	3
1.3 Functional Requirements	3
1.4 Use Case	4
1.4.1 Overview Diagrams	4
1.4.2 Internal Steps	4
2 Storyboard.....	5
3 Design	7
3.1 VOPC Diagram	7
3.2 Design Diagram MVC	8
3.3 Design Pattern.....	10
3.4 Activity Diagram	12
3.5 Sequence Diagram	13
3.6 State Diagram.....	14
4 Testing.....	15
5 Exception.....	16
6 Persistenza	17
6.1 File System	17
6.2 DBMS	17
7 Sonar Cloud e informazioni finali	18
8 Dizionario dei termini.....	19

1 SRS (Software Requirements Specification)

1.1 Introduzione

1.1.1 Scopo del documento

Lo scopo del presente documento è quello di fornire una descrizione chiara e strutturata dei principali requisiti funzionali e delle caratteristiche del sistema **SportBooking**, in modo da supportare la comprensione del sistema sia dal punto di vista dell'utente sia dal punto di vista dello sviluppo, fornendo una visione completa delle sue funzionalità e della sua struttura.

1.1.2 Descrizione generale del sistema

L'applicazione consente agli utenti di visualizzare i campi sportivi disponibili, prenotare specifici slot temporali in base alla disponibilità e richiedere la disdetta di prenotazioni esistenti al gestore.

Il sistema supporta inoltre la gestione degli account e l'applicazione di penalità in caso di comportamenti non corretti.

Sono inoltre previste funzionalità dedicate al gestore degli impianti sportivi, che può controllare la disponibilità dei campi, configurare le regole del sistema e gestire operazioni relative agli utenti.

L'obiettivo principale del sistema è semplificare l'organizzazione delle attività sportive, fornendo una piattaforma centralizzata che garantisca flessibilità, coerenza e facilità d'uso sia per i clienti sia per i gestori.

1.1.3 Hardware and Software Requirements

Il sistema richiede un elaboratore in grado di eseguire applicazioni Java.

Sono necessari un sistema operativo compatibile e Java Development Kit (JDK 17 o superiore).

Per la modalità con persistenza su database è richiesto un DBMS relazionale (es. MySQL).

Non sono richieste particolari specifiche hardware oltre a quelle necessarie per l'esecuzione della JVM.

1.1.4 Related Systems

Playtomic

Un sistema simile a **SportBooking** è **Playtomic**, una piattaforma utilizzata per la prenotazione di campi sportivi, soprattutto per sport come padel e tennis. Tra i vantaggi principali vi sono un'interfaccia intuitiva e una larga diffusione che consente di accedere a numerosi impianti sportivi. Tuttavia, presenta alcuni limiti legati alla ridotta flessibilità nella gestione delle regole di prenotazione e alla presenza di funzionalità avanzate accessibili solo tramite servizi a pagamento.

Sportclub Manager

Un altro sistema correlato è **Sportclub Manager**, pensato per la gestione completa di centri sportivi e associazioni. Questo sistema offre una gestione avanzata degli utenti e delle prenotazioni, risultando particolarmente adatto per contesti organizzativi complessi. D'altro canto, l'interfaccia risulta meno intuitiva per utenti inesperti e l'utilizzo complessivo può risultare più complesso rispetto a soluzioni più semplici.

1.2 User Stories

1) Come cliente,

voglio visualizzare la disponibilità dei campi,

così da poter scegliere l'orario e il giorno più adatti ai partecipanti.

2) Come cliente,

voglio disdire una prenotazione,

così da annullare la prenotazione ed evitare eventuali penalità.

3) Come gestore,

voglio visualizzare le richieste di disdetta,

così da approvarle o rifiutarle e liberare gli slot per altri clienti.

1.3 Functional Requirements

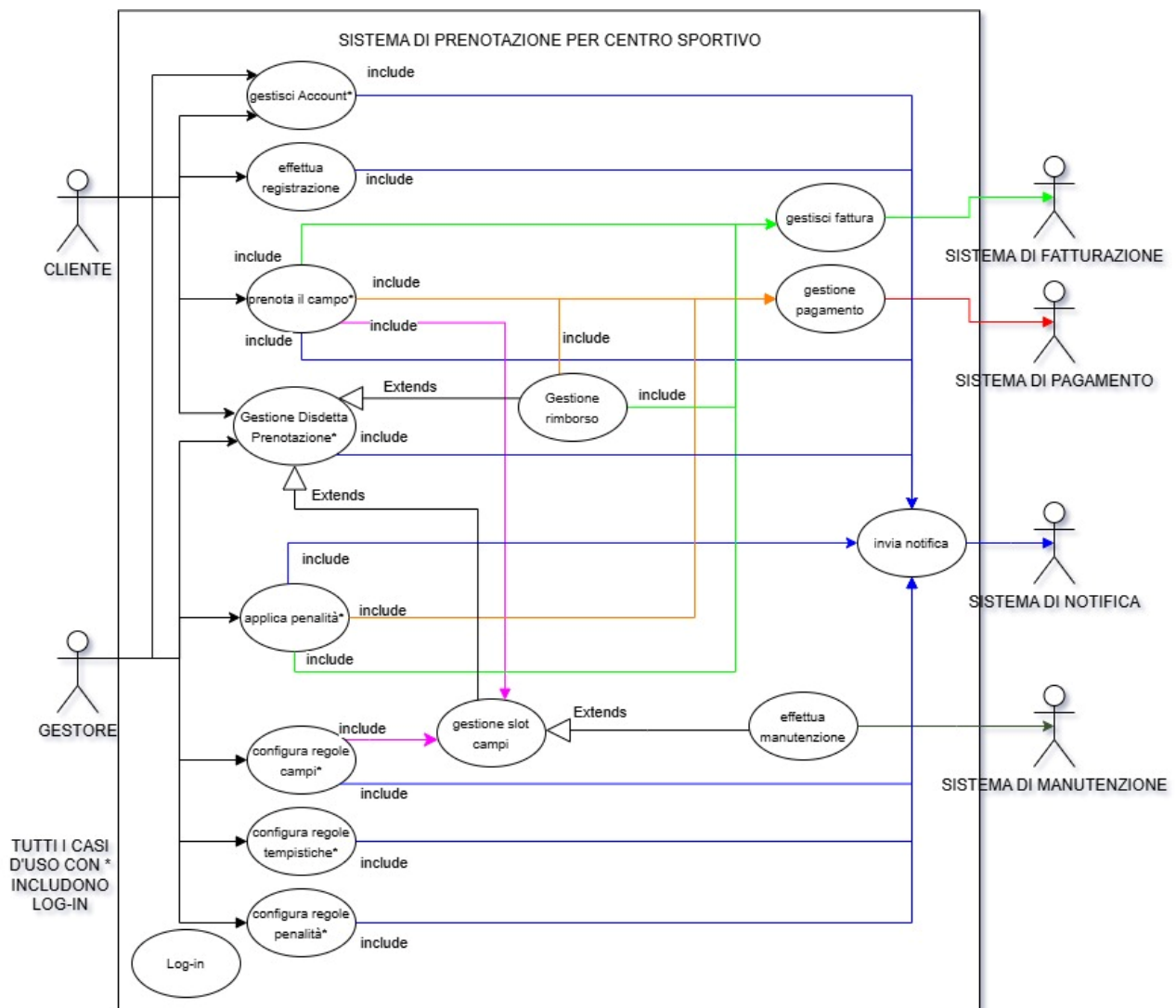
1) Il sistema deve registrare un nuovo utente tramite l'inserimento dei dati richiesti.

2) Il sistema deve fornire gli slot disponibili in base al campo, alla data e alla fascia oraria selezionati.

3) Il sistema deve registrare una richiesta di disdetta per una prenotazione esistente.

1.4 Use Case

1.4.1 Overview Diagrams



1.4.2 Internal Steps

Effettua Registrazione

1. Il Cliente richiede di effettuare la registrazione al sistema.
2. Il Sistema richiede al Cliente i dati necessari per creare un nuovo account.
3. Il Cliente inserisce i dati richiesti.
4. Il Sistema verifica la validità dei dati inseriti.
5. Il Sistema verifica che l'e-mail inserita non sia già associata a un account esistente.
6. Il Sistema crea il nuovo account del Cliente.
7. Il Sistema notifica l'avvenuta registrazione al Cliente.

Estensioni

4a) La password inserita contiene meno di sei caratteri: il Sistema informa il Cliente che la password deve contenere almeno sei caratteri e richiede una nuova compilazione.

4b) L'e-mail inserita non rispetta il formato richiesto: il Sistema informa il Cliente che l'e-mail inserita non è valida e richiede una nuova compilazione.

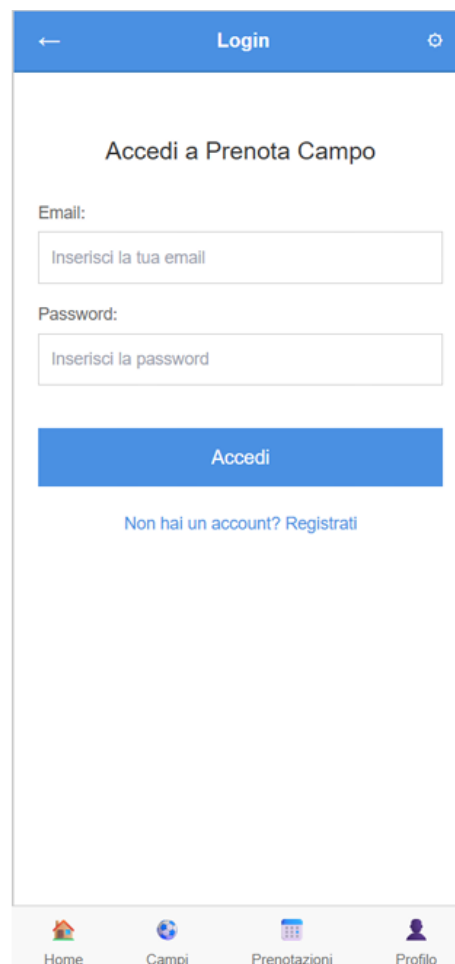
5a) L'e-mail inserita è già associata a un account esistente: il Sistema informa il Cliente che l'e-mail è già in uso e termina la registrazione senza creare un nuovo account.

2 Storyboard

Prototipi dell'applicazione che hanno lo scopo di fare vedere al cliente il risultato futuro e capire se è quello che vuole davvero

Log-in

La prima storyboard rappresenta la schermata di log-in, nella quale il Cliente inserisce le proprie credenziali per accedere ai servizi dell'applicazione.



The image shows a mobile app login screen prototype. At the top is a blue header bar with a back arrow on the left, the word "Login" in the center, and a settings gear icon on the right. Below the header, the text "Accedi a Prenota Campo" is centered. Underneath, there are two input fields: the first is labeled "Email:" and contains the placeholder text "Inserisci la tua email"; the second is labeled "Password:" and contains the placeholder text "Inserisci la password". Below these fields is a large blue button with the text "Accedi". Under the button, there is a link that says "Non hai un account? Registrati". At the bottom of the screen is a navigation bar with four icons and labels: a house icon for "Home", a globe icon for "Campi", a calendar icon for "Prenotazioni", and a person icon for "Profilo".

Disdici Prenotazione

La seconda storyboard rappresenta la schermata di richiesta disdetta, nella quale il Cliente visualizza le prenotazioni confermate e seleziona quella per cui avviare la richiesta.



3 Design

In questa sezione vengono analizzate le principali caratteristiche interne del sistema, attraverso l'utilizzo di diagrammi scelti per rappresentare in maniera complementare il dominio del problema e quello della soluzione.

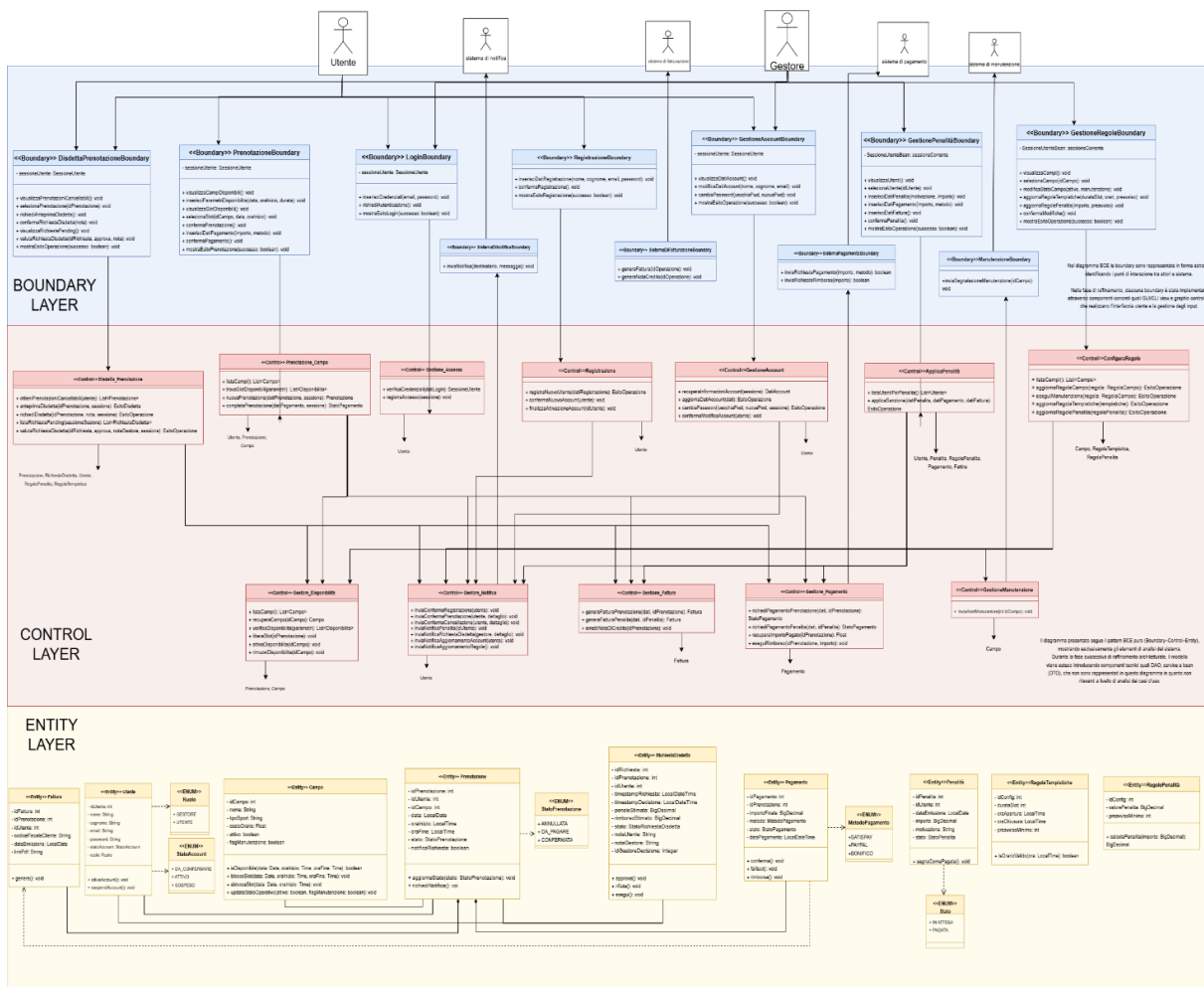
Scelta dei diagrammi

Per la rappresentazione del sistema sono stati utilizzati due diagrammi distinti:

Diagramma BCE (Boundary–Control–Entity) per il VOPC (View of Participating Classes), che descrive il sistema dal punto di vista del **dominio del problema**;

Diagramma MVC (Model–View–Controller) per il design architetturale, che rappresenta il sistema nel **dominio della soluzione**.

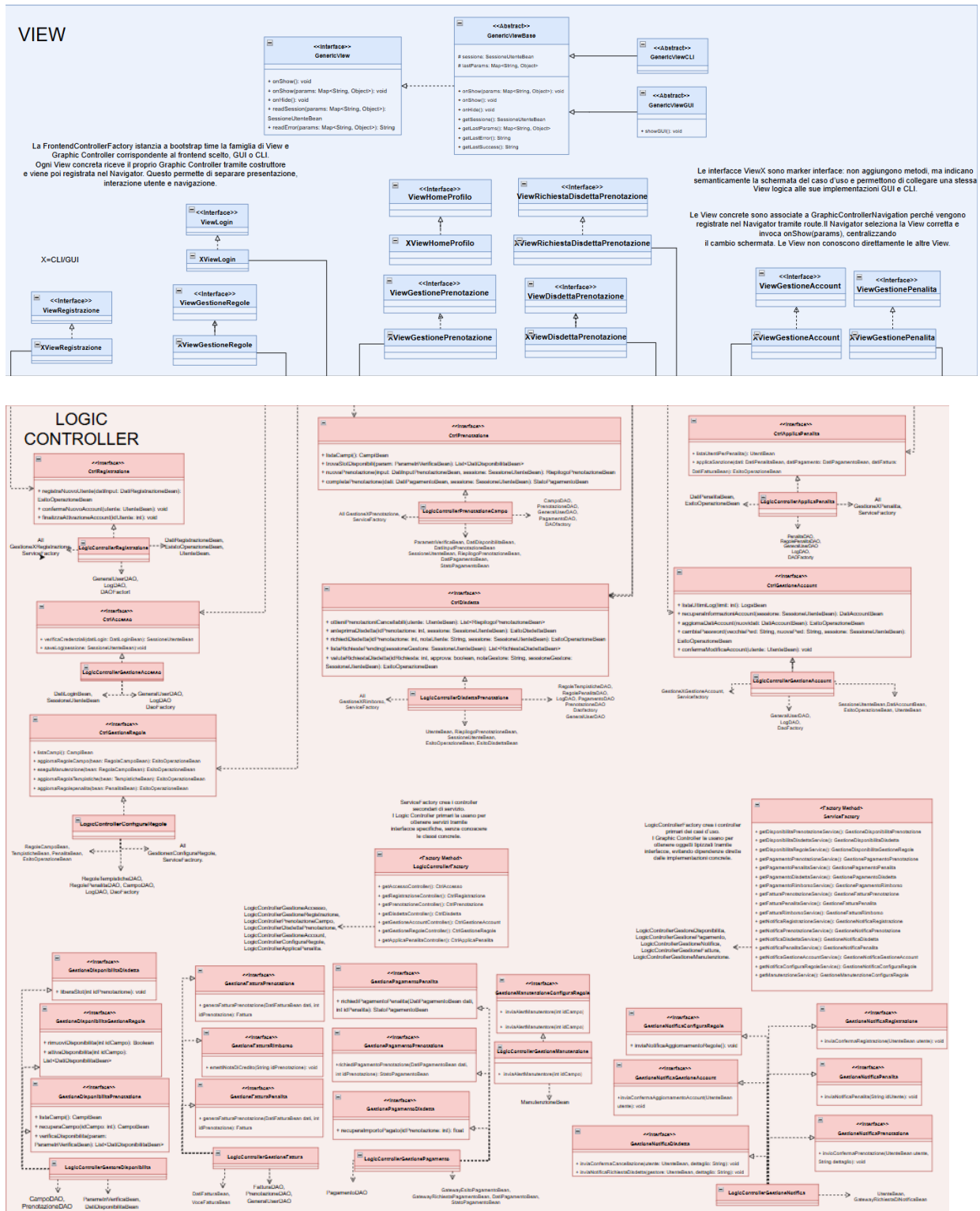
3.1 VOPC Diagram

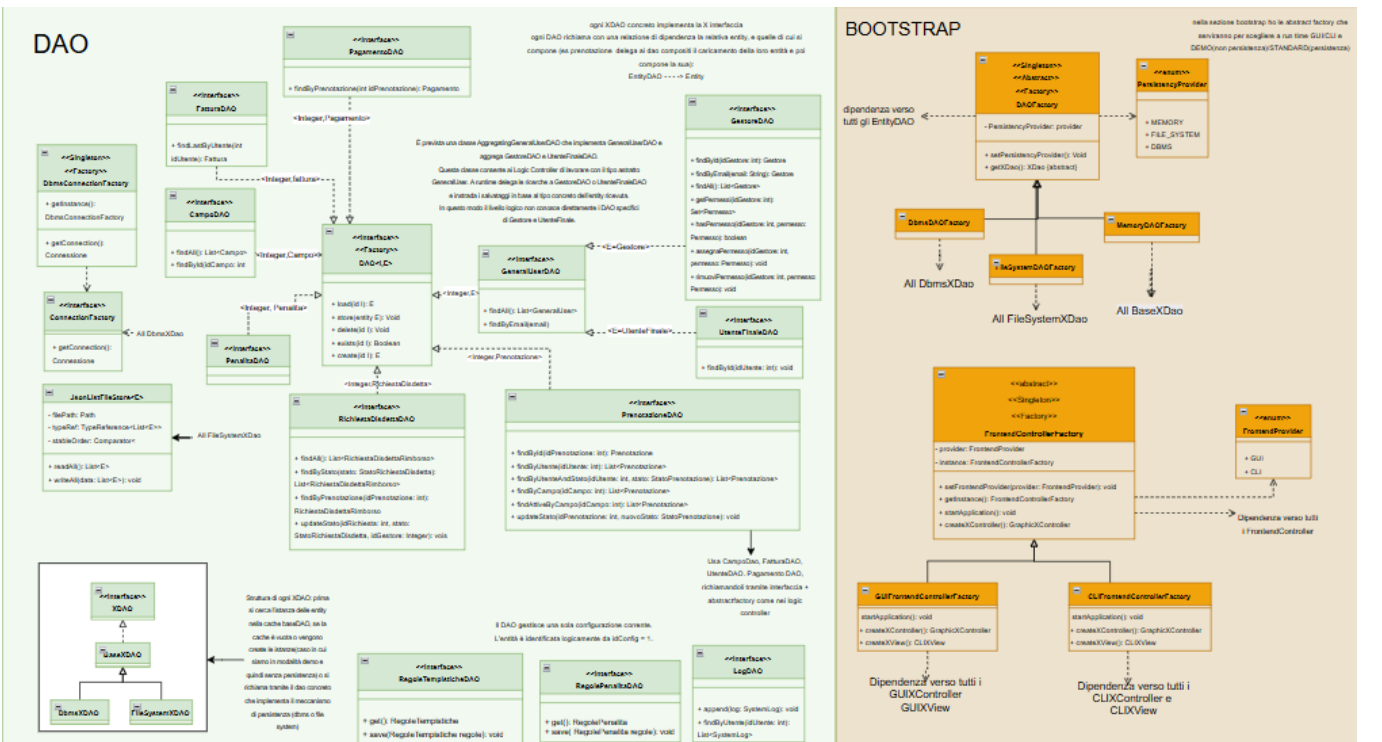


Link per osservare il diagramma approfonditamente: https://drive.google.com/file/d/1S-FftRCxYOhci41rJWVZEQ8dez_n9PZu/view?usp=sharing

3.2 Design Diagram MVC

Mentre il diagramma BCE rappresenta il sistema a livello di analisi, il diagramma MVC descrive la struttura concreta dell'implementazione, evidenziando i componenti software e le dipendenze tra i vari layer.





In questo modo i controller logici non dipendono dalle implementazioni concrete dei DAO, ma soltanto dalle interfacce, di conseguenza, la logica applicativa rimane invariata indipendentemente dal tipo di persistenza scelto. Il controller richiede un DAO alla factory e riceve un oggetto compatibile con l'interfaccia attesa, senza conoscere se tale oggetto lavori su database, filesystem o memoria.

Lo stesso principio è stato applicato al layer di presentazione tramite `FrontendControllerFactory`, che permette di selezionare la famiglia di componenti grafici in base alla scelta GUI o CLI.

Le factory concrete creano i componenti coerenti con il frontend selezionato, cioè:

- i **Graphic Controller** specifici per GUI o CLI;
- le **View** concrete;
- il **Navigator** utilizzato per gestire il cambio di schermata.

Questa soluzione mantiene separata la logica applicativa dalla tecnologia di presentazione. I controller logici non dipendono dalla GUI o dalla CLI, mentre le view e i controller grafici vengono creati in modo coerente con il frontend scelto.

Oltre all'**Abstract Factory**, nel progetto è stato utilizzato anche il pattern **Singleton** per garantire che alcune factory vengano inizializzate una sola volta durante l'esecuzione dell'applicazione.

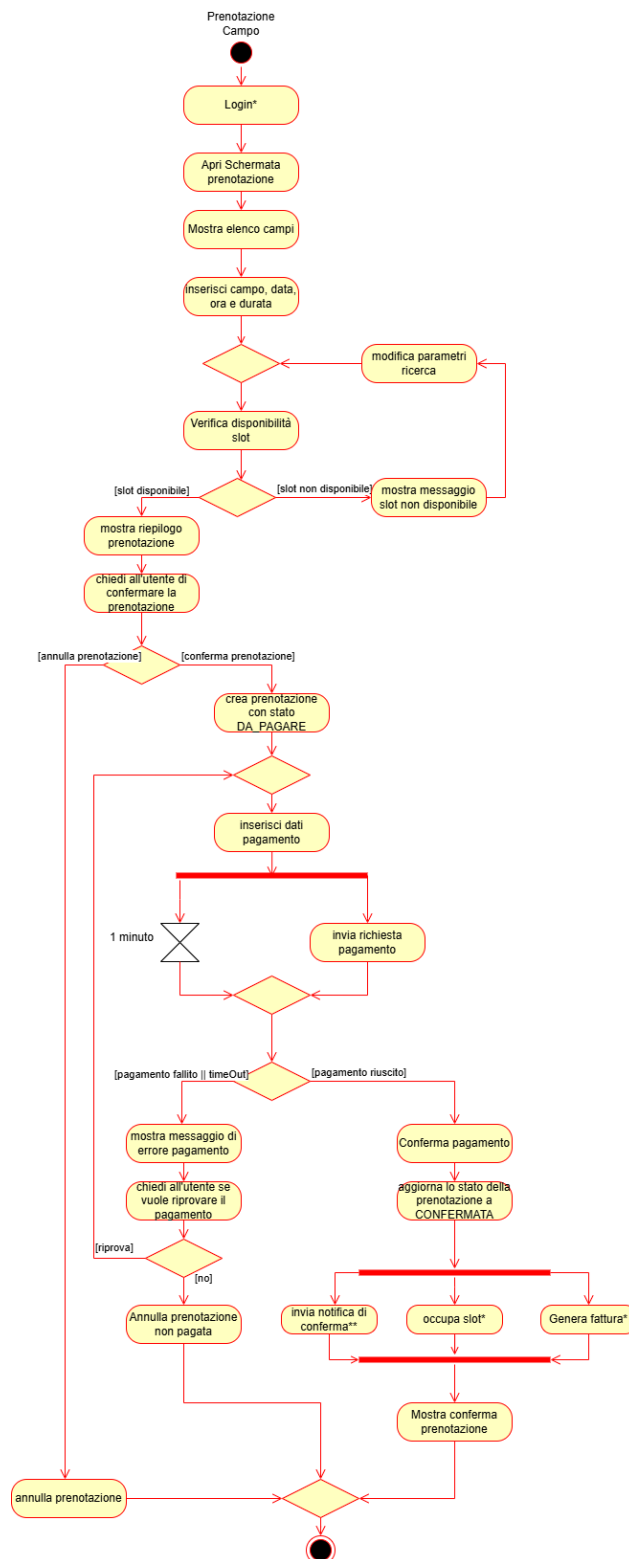
Questo aspetto è importante perché il provider di persistenza e il frontend vengono scelti all'avvio del sistema e non devono cambiare a runtime. In questo modo si evitano configurazioni incoerenti, ad esempio una parte dell'applicazione che usa il provider DBMS e un'altra che usa il provider FileSystem, oppure componenti GUI e CLI mescolati nella stessa esecuzione.

In sintesi, l'utilizzo combinato di **Abstract Factory** e **Singleton** permette di ottenere:

- disaccoppiamento dalle implementazioni concrete;
- selezione controllata di frontend e persistenza;
- maggiore estendibilità del sistema;
- configurazione coerente per tutta l'esecuzione dell'applicazione.

3.4 Activity Diagram

Il seguente Activity Diagram descrive il flusso di prenotazione di un campo, includendo disponibilità, conferma, pagamento e gestione dei casi alternativi.

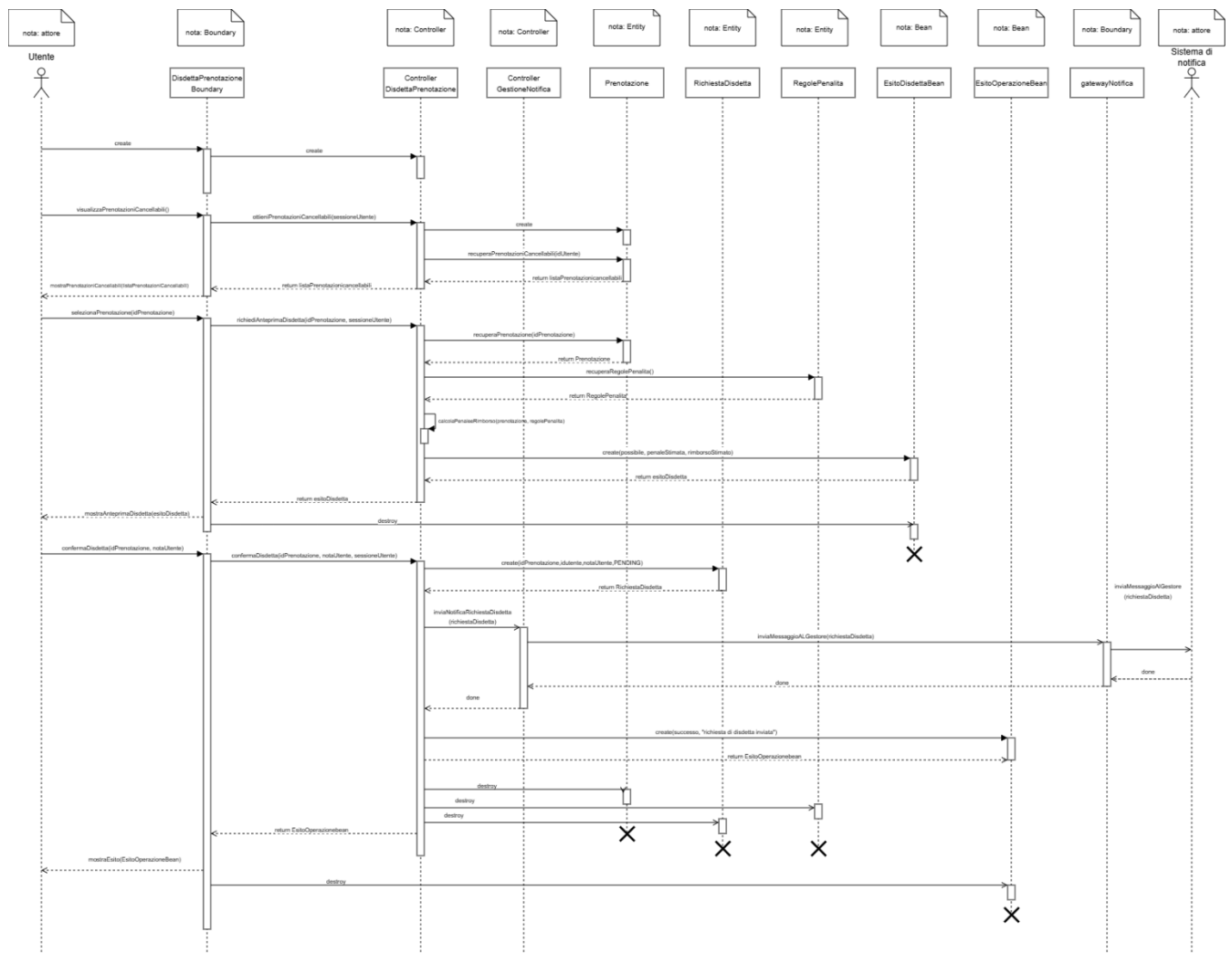


Link per osservare il diagramma in modo più approfondito:

<https://drive.google.com/file/d/1757EN3o7xa1rxBJnrtzzz3vwko1lTZGG/view?usp=sharing>

3.5 Sequence Diagram

Il Sequence Diagram descrive le interazioni BCE del caso d'uso "Richiesta disdetta prenotazione", mostrando il passaggio dei messaggi tra utente, boundary, controller, entity, bean e sistema di notifica.

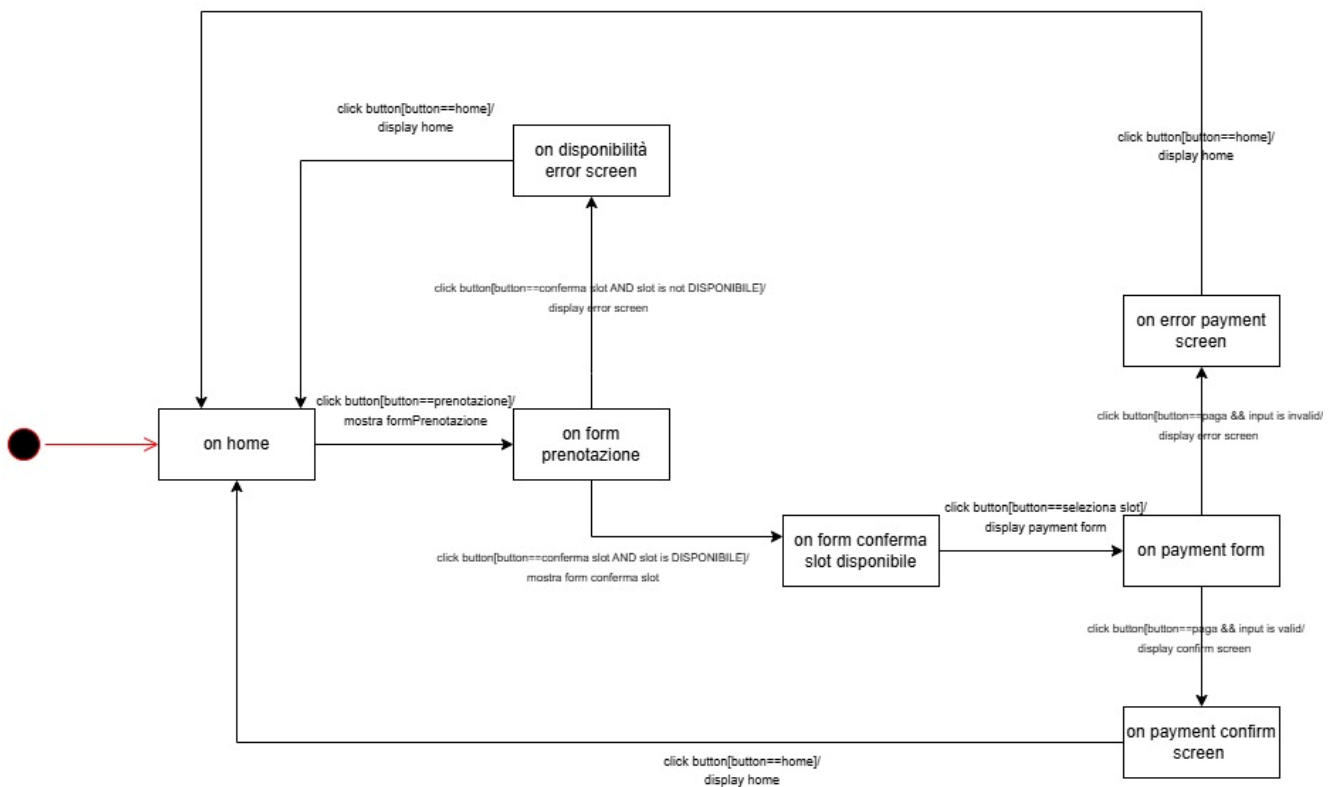


Link per osservare il diagramma in modo più approfondito:

<https://drive.google.com/file/d/1731dYbpW4hZmV6FLvAsSL51Pi2KB5cbp/view?usp=sharing>

3.6 State Diagram

Lo State Diagram descrive gli stati principali della prenotazione campo, includendo form di prenotazione, conferma slot, pagamento, errori e ritorno alla home.



Link per osservare il diagramma in modo più approfondito: <https://drive.google.com/file/d/1K-yMT1lN49ylFmfpz0a-zupzPaJcxBuE/view?usp=sharing>

4 Testing

Nella parte di testing sono state sviluppate diverse classi di test per verificare il comportamento dei principali controller logici dell'applicazione SportBooking. I test si concentrano soprattutto sulla validazione degli input, sulla gestione dei casi di errore e sull'orchestrazione tra controller logici e servizi applicativi.

LogicControllerApplicaPenalitaTest

Questa classe verifica alcuni casi di errore del controller **LogicControllerApplicaPenalita**, controllando il comportamento del sistema quando i dati della penalità sono nulli, quando l'id dell'utente non è valido e quando la motivazione della penalità non rispetta i vincoli richiesti. In tutti questi casi il test verifica che il risultato non sia nullo e che l'operazione termini con esito negativo.

LogicControllerGestioneAccountTest

Questa classe verifica alcune funzionalità del controller **LogicControllerGestioneAccount**, tra cui il recupero delle informazioni dell'account, il cambio password e l'aggiornamento dei dati dell'account. I test coprono casi come sessione nulla, utente non presente nella sessione, e-mail vuota, password non valida e dati account non corretti. In questi scenari viene verificato che il controller restituisca un risultato coerente con l'errore rilevato.

LogicControllerApplicaPenalitaMockitoTest

Questa classe utilizza **Mockito** per verificare l'orchestrazione del caso d'uso di applicazione penalità. In particolare, vengono simulati i servizi di pagamento, fatturazione e notifica, controllando che il controller invochi correttamente i collaboratori previsti. Il test verifica inoltre che i dati di pagamento vengano normalizzati correttamente e che i servizi vengano chiamati il numero corretto di volte.

Questi test coprono sia scenari di errore sia scenari di collaborazione tra componenti, verificando la correttezza del comportamento dei controller logici.

5 Exception

Nella parte relativa alla gestione delle eccezioni, il sistema definisce classi dedicate per gestire scenari di errore specifici del caso d'uso “**Effettua registrazione**”. Questa scelta rispetta il requisito di progetto secondo cui le eccezioni devono essere gestite correttamente, senza limitarsi alla semplice cattura e propagazione.

Le eccezioni dedicate ereditano dalla classe base **RegistrationException**, che raggruppa gli errori relativi al processo di registrazione. Queste eccezioni vengono utilizzate da **LogicControllerRegistrazione** per verificare che i dati inseriti dall'utente rispettino i vincoli richiesti.

PasswordTooShortException

Viene sollevata quando la password inserita contiene meno di sei caratteri. In questo caso la registrazione non viene completata e il sistema informa l'utente che la password deve contenere almeno sei caratteri.

InvalidEmailFormatException

Viene sollevata quando l'e-mail inserita non rispetta il formato previsto. In questo caso il sistema informa l'utente che il formato dell'e-mail non è valido e richiede un indirizzo corretto.

EmailAlreadyExistsException

Viene sollevata quando l'e-mail inserita è già associata a un account esistente. In questo caso il sistema interrompe il processo di registrazione e informa l'utente che l'e-mail è già in uso.

Queste eccezioni non vengono semplicemente catturate e rilanciate. Sono gestite dai controller grafici, che adattano il messaggio di errore all'interfaccia selezionata. Nella versione GUI il messaggio viene mostrato nella schermata di registrazione, mentre nella versione CLI il messaggio viene stampato in console. Questo approccio mantiene separata la logica applicativa dal livello di presentazione e rende espliciti i flussi alternativi del caso d'uso.

6 Persistenza

Il sistema **SportBooking** supporta diverse modalità di gestione dei dati.

La scelta del provider di persistenza viene effettuata in fase di bootstrap e rimane invariata durante l'esecuzione dell'applicazione.

La persistenza è gestita tramite il pattern DAO, che separa la logica applicativa dai dettagli tecnici di accesso ai dati.

In questo modo i controller lavorano con interfacce senza conoscere se i dati siano salvati su file JSON, su database relazionale o mantenuti solo in memoria.

Nel progetto la persistenza compare principalmente in due forme: I/O su filesystem JSON e I/O su DBMS tramite JDBC.

6.1 File System

Nella modalità **File System**, i dati del sistema vengono salvati in file JSON. Ogni DAO concreto del provider filesystem gestisce il file relativo alla propria entity, ad esempio: **campi.json**, **prenotazioni.json**, **pagamenti.json**, **fatture.json**.

La classe centrale per l'I/O su file JSON è **JsonListFileStore**. Questa classe fornisce due operazioni principali: `readAll()`, `writeAll()`.

`readAll()` legge il contenuto del file JSON e lo converte in una lista di oggetti Java, mentre `writeAll()` prende una lista di oggetti Java e la salva nel relativo file JSON. La conversione tra oggetti Java e JSON viene effettuata tramite **ObjectMapper** di Jackson.

Ad esempio, per cercare una prenotazione per ID, il DAO legge l'intero file **prenotazioni.json**, costruisce una mappa indicizzata per `idPrenotazione` e restituisce l'elemento richiesto. Allo stesso modo, metodi come `findByCampo`, `findByUtente` o `findByUtenteAndStato` leggono tutte le prenotazioni e filtrano la lista tramite codice Java.

Il vantaggio principale di questa soluzione è la semplicità: i dati sono leggibili e facilmente ispezionabili nei file JSON.

6.2 DBMS

Nella modalità **DBMS**, i dati vengono salvati in un database relazionale, ad esempio **MySQL**.

In questo caso i DAO concreti comunicano con il database tramite **JDBC**.

JDBC permette a un'applicazione Java di interagire con un DBMS relazionale attraverso una API comune.

Nel progetto, il flusso tipico di un DAO DBMS prevede l'apertura di una `Connection`, la creazione di un `PreparedStatement`, l'esecuzione della query SQL e la lettura del risultato tramite `ResultSet`, infine il `ResultSet` viene poi mappato in oggetti Java del model.

7 Sonar Cloud e informazioni finali

Sonar Cloud

Sono stati eseguiti dei controlli di qualità del codice tramite Sonar Cloud, con l'obiettivo di individuare e ridurre il numero di code smell, siccome ciò incide molto sulla manutenibilità del prodotto software:

https://sonarcloud.io/summary/new_code?id=enricobarbatano_Progetto_ISPW

Nel report SonarCloud è presente una duplicazione limitata, concentrata soprattutto nel layer DAO. Tale duplicazione è stata mantenuta consapevolmente perché le implementazioni FileSystem e DBMS utilizzano meccanismi di persistenza differenti.

È stata valutata l'introduzione di classi astratte o utility comuni per ridurre la duplicazione, ma questa soluzione avrebbe aumentato l'accoppiamento tra implementazioni tecnicamente diverse e avrebbe reso meno chiara la responsabilità dei singoli DAO.

Per questo motivo è stata accettata una duplicazione controllata, preservando il principio SRP e l'indipendenza tra i diversi provider di persistenza.

GitHub

Per il progetto è stato usato il sistema di versionamento Git con commit frequenti e repository pubblico su GitHub:

https://github.com/enricobarbatano/Progetto_ISPW

Youtube

Per dimostrare le funzionalità principali del sistema è stato predisposto un breve video dimostrativo caricato sia su YouTube che su Google Drive:

- link di youtube:
<https://youtu.be/CC-ZvJL1i8k?si=k9JEX4PZU3ckWnBc>
- link di google drive:
https://drive.google.com/file/d/1-k7ZenJfgM_clhZukmkUf81lBWBZq6AD/view?usp=sharing

8 Dizionario dei termini

In questa sezione vengono chiariti alcuni termini utilizzati nella documentazione, al fine di evitare ambiguità tra concetti di analisi, design e implementazione.

Utente / Cliente

Nel documento i termini Utente e Cliente vengono usati come sinonimi.

Entrambi indicano il soggetto che utilizza l'applicazione per visualizzare la disponibilità dei campi, effettuare prenotazioni e richiedere la disdetta di una prenotazione.

Gestore

Il Gestore indica l'utente con funzionalità amministrative sul sistema.

Nel progetto il gestore può visualizzare richieste di disdetta, approvarle o rifiutarle, configurare regole e gestire operazioni relative agli utenti o ai campi sportivi.

DAOFactory

DAOFactory è la factory responsabile della selezione della famiglia di DAO da utilizzare.

La scelta avviene in fase di bootstrap in base al provider di persistenza configurato, mantenendo i controller indipendenti dalle implementazioni concrete dei DAO.

Provider

Un Provider indica una modalità concreta selezionata per una parte variabile del sistema.

Nel documento il termine è usato soprattutto per indicare la modalità di persistenza o la modalità di frontend scelte all'avvio dell'applicazione.

JDBC

JDBC è l'API Java utilizzata per comunicare con un database relazionale.

Nel progetto viene citata nella persistenza per descrivere l'uso di Connection, PreparedStatement e ResultSet nei DAO che comunicano con il database.

ObjectMapper

ObjectMapper è il componente della libreria Jackson utilizzato per convertire oggetti Java in formato JSON e viceversa.

Nel progetto viene citato nella persistenza per descrivere la serializzazione e deserializzazione degli oggetti nei file JSON.

Service Controller

Un Service Controller è un controller secondario che gestisce funzionalità di supporto rispetto ai casi d'uso principali.

Nel progetto questo termine riguarda componenti legati, ad esempio, a pagamento, fatturazione, notifiche, disponibilità o manutenzione

FrontendControllerFactory

FrontendControllerFactory è la factory responsabile della creazione dei componenti del frontend selezionato.

Le factory concrete creano View, Graphic Controller e Navigator coerenti con la modalità di interfaccia scelta in fase di bootstrap.

Navigator

Il Navigator è il componente che gestisce il cambio di schermata.

Mantiene l'associazione tra una route e una View e richiama la schermata corretta durante la navigazione.

Gateway

Un Gateway rappresenta un punto di comunicazione con un sistema esterno.

Nel progetto viene usato per separare la logica applicativa dal sistema esterno che esegue materialmente un'operazione, ad esempio l'invio di una notifica.

Bootstrap

Il Bootstrap indica la fase iniziale di configurazione e avvio dell'applicazione.

In questa fase vengono selezionati provider, frontend e componenti principali necessari all'esecuzione del sistema.

SRP (Single Responsibility Principle)

Indica il principio secondo cui una classe dovrebbe avere una responsabilità principale ben definita. Nel documento viene richiamato soprattutto per giustificare la separazione delle responsabilità nel layer DAO.

DIP (Dependency Inversion Principle)

Indica il principio secondo cui i componenti di alto livello dovrebbero dipendere da astrazioni e non da implementazioni concrete. Nel progetto questo principio è applicato tramite interfacce, DAO e factory.